

PARKING LOT COUNTER SYSTEM

DIGITAL LOGIC DESIGN

MEMBERS:

ABUBAKER (L)	41589
ASIF ULLAH	41695
SAIF QAMAR	41492
JAM AWAIS	41567

Submitted to:

MS. SAFINA

DATED:

January 16, 2025

PARKING LOT COUNTER SYSTEM

Problem Statement:

Design and implement a digital system to monitor and control the number of cars entering and exiting a parking lot. The system should display the current count of vehicles and provide an alert when the maximum parking capacity is reached (e.g., 9 vehicles).

SOFTWARE IMPLEMENTATION

Apparatus Required:

1. Integrated Circuits (ICs):

- 74LS192D (Up/Down Counter)
- 74LS47D (BCD to 7-Segment Decoder)

2. Passive Components:

- Resistors
- LED (Indicator)
- Buzzer (400 Hz)

3. Power Supply: 10V DC

4. Switches:

- Entry Point (S1)
- Exit Point (S2)

5. 7-Segment Display (Common Anode)

6. Multisim Software for Simulation

Details of Components (ICs):

1. 74LS192D: Up/Down Counter:

- **Function:** This IC is the **heart of the parking counter system**. It **increments** or **decrements** the count based on the input signals from S1 (entry) and S2 (exit).
- **Key Features:**
 - Count range: **0-9** (configured for decimal counting)
 - Clear and Load inputs for **resetting and initializing**

2. 74LS47D: BCD to 7-Segment Decoder:

- **Function:** Converts the binary count from **74LS192D to a 7-segment display** format for visual representation.
 - **Key Features:**
 - Drives common anode 7-segment displays
 - Includes blanking and lamp test features
-

IC Pin Configuration:

74LS192D (Up/Down Counter):

Pin Number	Function
5	Count up Input
4	Count down Input
3-7	Binary Inputs
12	Count Up Input
11	Load input
13	Borrow pin

12	Carry pin
----	-----------

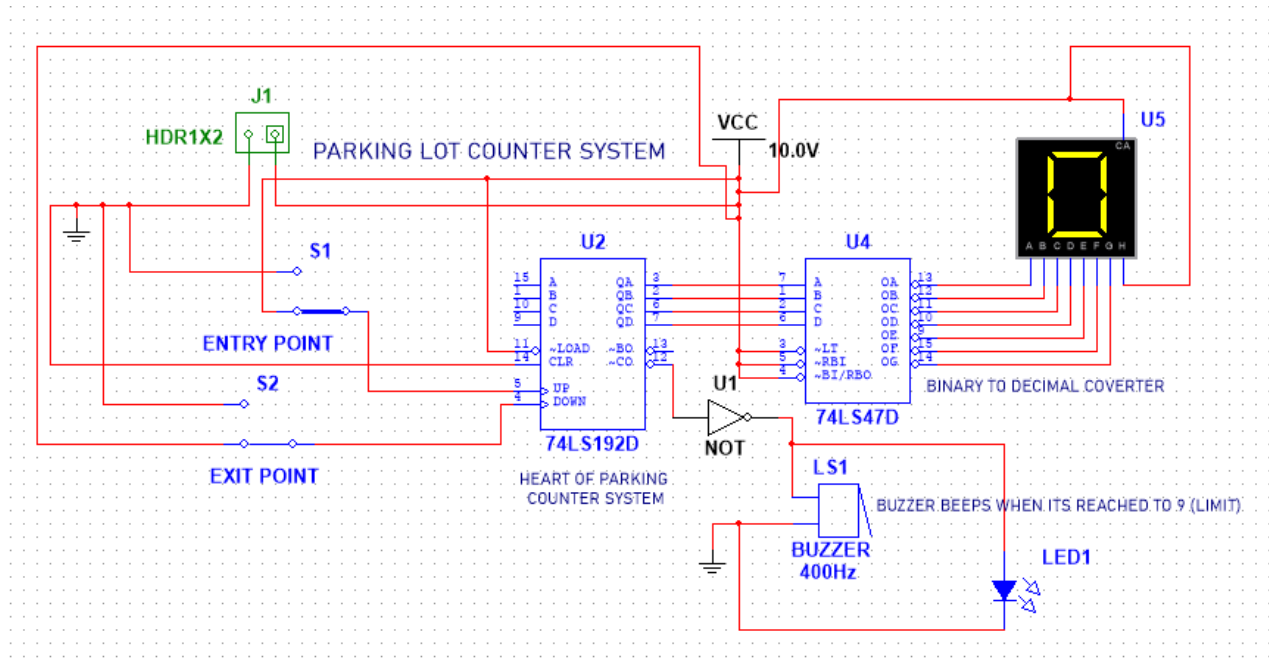
74LS47D (BCD to 7-Segment Decoder):

Pin Number	Function
1-4	BCD Inputs
3	Lamp Test
5	Ripple Blanking
9-15	Segment Outputs
4	Blanking input

Multisim Simulation Screenshots:

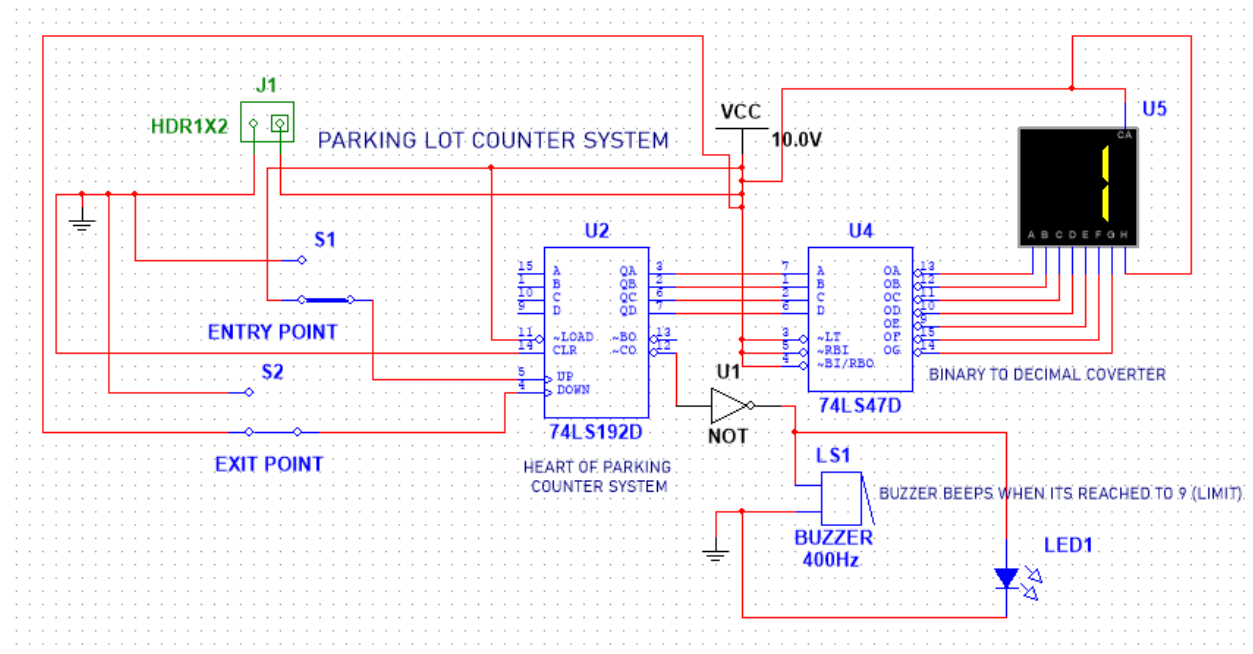
1. Screenshot:

- Confirming that **no vehicle** has **entered** so far.



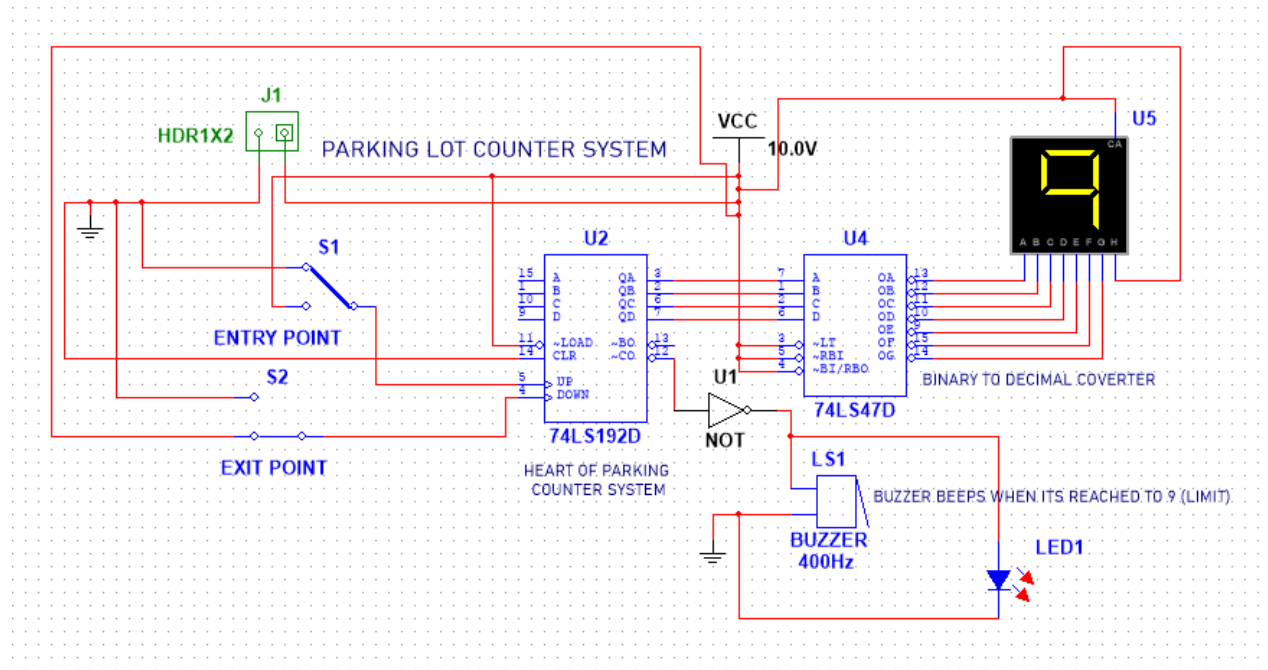
2. Screenshot:

- Confirming the **entry** of **one** vehicle.



3. Screenshot:

- Indicating that the **parking lot is full (0-9)** with an LED activation and a buzzer sound.



Truth Table:

The truth table for the **74LS192D** counter:

Input (Count Up/Down)	Binary Output (Q3 Q2 Q1 Q0)	Decimal Output
Count Up	0001	1
Count Up	0010	2
Count Down	0001	1
Count Down	0000	0

Result:

The Parking Lot Counter System successfully **tracks** the number of **vehicles entering and exiting** the parking lot. The current **count** is **displayed** on a **7-segment display**. When the **count reaches** the maximum capacity (**9 vehicles**), a **buzzer** is activated to indicate no further entry is allowed. The system can **reset** to **zero** for reuse.

HARDWARE IMPLEMENTATION

Introduction:

Hardware implementation of an automated parking lot counter system designed to manage vehicle **entry and exit while keeping track of available parking slots**. The system ensures **efficient space** utilization and **denies entry** when the **parking** lot is **full**.

Apparatus Required:

1. **Arduino UNO**: Acts as the **main controller**, managing inputs from sensors and controlling outputs.
 2. **IR Proximity Sensors**:
 - **Entry Sensor**: Detects vehicles approaching the **barrier for entry**.
 - **Exit Sensor**: Detects vehicles **leaving the parking** lot after the barrier.
 3. **Servo Motor**: Controls the **barrier arm**, raising or lowering it based on the sensor input.
 4. **16x2 LCD I2C Display**: Displays the **number** of **available parking slots** or a message such as "Sorry, Lot is Full."
 5. **Jumpers and Breadboard**: Used to **connect** the **components** in the circuit.
-

Working Mechanism:

1. Entry Process:

- When a vehicle approaches the entry barrier, the **entry IR sensor** detects it.
- The **Arduino UNO** processes the **sensor signal** and sends a command to the **servo motor**, causing the barrier to raise.
- The vehicle enters the parking lot, and the counter on the **LCD display** decreases by 1, indicating the number of remaining slots.

2. Exit Process:

- When a vehicle exits, the **exit IR sensor** detects it.
- The **Arduino UNO** processes the signal and commands the **servo motor** to raise the barrier.
- The counter on the **LCD display** increases by 1, updating the number of available slots.

3. Slot Availability:

- The system starts with 5 slots available (Total = 5).
 - Each time a vehicle enters, the counter decreases (e.g., 5 → 4 → 3 → ...).
 - When the counter reaches **0**, the **LCD display** shows the message: **"Sorry, Lot is Full"**, and no vehicle is allowed to enter.
 - When a vehicle exits, the counter updates accordingly, and entry is permitted again.
-

System Flowchart:

1. Vehicle approaches **entry sensor**.
 2. IR sensor **detects** vehicle.
 3. Arduino **checks** if slots are available.
 - If yes, servo motor **raises the barrier**, vehicle enters, and the counter decreases.
 - If no, display shows "**Sorry, Lot is Full,**" and the barrier remains closed.
 4. Vehicle reaches **the exit sensor**, which detects it.
 5. Servo motor **raises the exit barrier**, vehicle exits, and the counter increases.
-

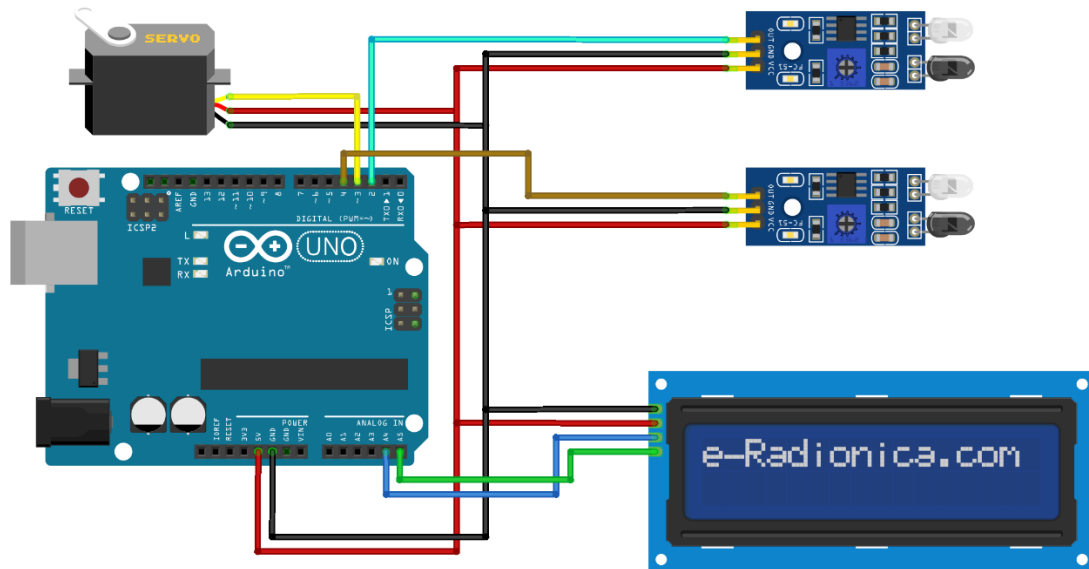
Hardware Circuit Connection:

The components are connected as follows:

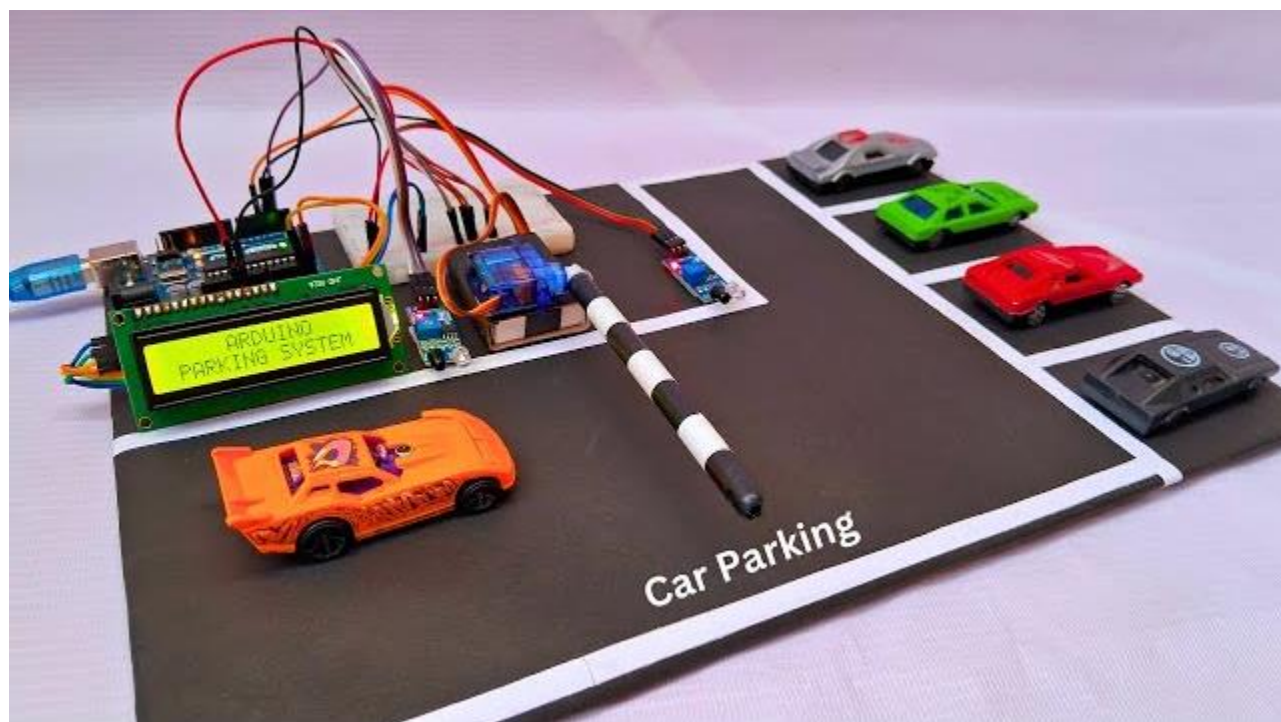
- **IR Sensors:** Connected to Arduino's digital **input pins**.
- **Servo Motor:** Connected to a **PWM pin** of Arduino for smooth movement.
- **LCD Display:** Connected via **I2C interface** for slot availability updates.

- **Power Supply:** **5V supply** from Arduino powers the sensors and display.

Hardware Circuit Diagram:



Hardware Picture:



Code in Arduino Software:

LIBRARIES

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h> //LCD AND IC2
```

```
#include <Servo.h> // used for motor or (barrier)
```

```
LiquidCrystal_I2C lcd(0x3F, 16, 2); // Change the HEX address if needed
```

```
Servo myservo1;
```

OBJECTS AND PINS:

```
int IR1 = 2; // Entry sensor //declare
```

```
int IR2 = 4; // Exit sensor
```

```
int totalSlots = 5; // Total number of parking slots
```

```
int usedSlots = 0; // Tracks used slots
```

FLAGS:

```
int flag1 = 0; // For entry sensor
```

```
int flag2 = 0; // For exit sensor
```

FUNCTION:

```
void setup()
```

```
{
```

FOR LCD:

```
  lcd.begin();
```

```
  lcd.backlight();
```

```
  pinMode(IR1, INPUT);
```

```
  pinMode(IR2, INPUT);
```

FOR BARRIER

```
  myservo1.attach(3); // Servo motor connected to pin 3
```

```
  myservo1.write(100); // Initially keep the barrier up
```

```
  lcd.setCursor(0, 0);
```

MAIN MESSAGE AFTER SWITCH ON

```
  lcd.print("  ARDUINO  ");
```

```
  lcd.setCursor(0, 1);
```

```
  lcd.print(" PARKING SYSTEM ");
```

```
  delay(2000);
```

```
lcd.clear();  
  
}
```

```
void loop()
```

```
{
```

CAR ENTRY DETECTION:

LOGIC HANDLING:

FOR ENTRY :

```
// Car entry detection (IR1)
```

```
if (digitalRead(IR1) == LOW && flag1 == 0) {
```

```
    if (usedSlots < totalSlots) { // Check if there's space
```

```
        flag1 = 1;
```

```
        if (flag2 == 0) {
```

```
            myservo1.write(0); // Raise barrier for car entry
```

```
            usedSlots += 1; //Changing here // Increment slots by 1 for each car
```

```
        if (usedSlots < totalSlots) {
```

```
            usedSlots = totalSlots; // Ensure it doesn't exceed total slots
```

```
        }
```

```
        delay(1000); // Wait for the car to enter
```

```
    }
```

```
} else {
```



```
lcd.setCursor(0, 0);  
lcd.print("  SORRY :(  ");  
lcd.setCursor(0, 1);  
lcd.print("  Parking Full  ");  
delay(3000); // Display for 3 seconds  
lcd.clear();  
}  
}
```

FOR EXIT:

```
// Car exit detection (IR2)  
if (digitalRead(IR2) == LOW && flag2 == 0) {  
    flag2 = 1;  
    if (flag1 == 0) {  
        myservo1.write(0); // Raise barrier for car exit  
        usedSlots -= 1;    // Decrease the used slots by 1  
        if (usedSlots < 0) {  
            usedSlots = 0; // Prevent usedSlots from going negative  
        }  
        delay(1000); // Wait for the car to exit  
    }  
}
```

```
}
```

IF WE BOTH CARS ENTERED AT SAME TIME BARRIER WILL NOT OPENS

```
// Reset the barrier after both sensors are triggered
```

```
if (flag1 == 1 && flag2 == 1)
```

```
{
```

```
  delay(1000); // Wait before lowering the barrier
```

```
  myservo1.write(100); // Lower the barrier
```

```
  flag1 = 0;
```

```
  flag2 = 0;
```

```
}
```

WELCOME AND SLOTS LEFT MESSAGE AFTER DISPLAYING ARDUINO PARKING SYSTEM

```
// Display the welcome message and available slots
```

```
lcd.setCursor(0, 0);
```

```
lcd.print("  WELCOME!  ");
```

```
lcd.setCursor(0, 1);
```

```
lcd.print("Slot Left: ");
```

```
lcd.print(totalSlots - usedSlots); // Display remaining available slots
```

```
delay(500); // Small delay to stabilize the display
```

```
}
```

Result:

The hardware implementation of the **parking lot counter system** effectively manages vehicle **entry and exit** while ensuring **no overcapacity**. The use of **Arduino UNO, IR sensors, servo motor, and an LCD display** allows for seamless operation. This system is scalable and can be adapted for **larger parking** lots by **increasing the number of slots and sensors**.

References:

Software Implementation:

URL: [Click here to view](#)

Hardware Implementation:

URL: [Click here to view](#)
